

# Quiz Game – Especificación de Arquitectura y Diseño (ADR)

Solución al Reto Técnico para el cargo  
Desarrollador de Software Senior.

**Preparado por:** Cristian Camilo Bonilla  
**Rol postulante:** Desarrollador de Software Senior  
**Empresa evaluadora:** CIEL INGENIERIA SAS  
**Fecha de entrega:** 11 de septiembre de 2026  
**Versión del documento:** v1.0

## Tabla de Contenido

|                                                          |   |
|----------------------------------------------------------|---|
| Estado de propuesta .....                                | 3 |
| Fecha .....                                              | 3 |
| Contexto .....                                           | 3 |
| Decisión.....                                            | 3 |
| Backend (.NET 10) .....                                  | 3 |
| Frontend (Angular 20).....                               | 4 |
| Pruebas y Calidad .....                                  | 4 |
| Alternativas consideradas .....                          | 4 |
| Consecuencias .....                                      | 4 |
| Positivas .....                                          | 4 |
| Negativas / limitaciones aceptadas .....                 | 5 |
| Plan de evolución (v1 → v2 / producción) .....           | 5 |
| Diagramas.....                                           | 5 |
| Modelo de Casos de Uso y Reglas de Negocio .....         | 5 |
| Detalle de los Casos de Uso .....                        | 6 |
| Arquitectura del Sistema (C4).....                       | 6 |
| Contenedores y Despliegue (C4 Nivel 1 y 2) .....         | 7 |
| Ciclo de Vida y Máquina de Estados .....                 | 7 |
| Modelo Relacional de Datos (SQL Server 2022).....        | 7 |
| Diccionario de Datos.....                                | 7 |
| Invariantes de Dominio vs Restricciones de Esquema ..... | 8 |
| Referencias .....                                        | 9 |

# Estado de propuesta

Aceptado

## Fecha

11 de septiembre de 2026

## Contexto

El sistema **QuizGame** requiere el desarrollo de una plataforma interactiva de preguntas y respuestas con dinámicas de progreso en tiempo real, gestión de aleatorios/acumulados y persistencia distribuida. Los requerimientos clave de la solución imponen:

- **Determinismo y autoridad en el servidor**  
El cliente debe comportarse exclusivamente como una vista reactiva. Cálculo de premios, temporizadores, validación de respuestas y el control de flujo de partidas residen 100% en el backend.
- **Comunicación bidireccional y resiliencia**  
Sincronización continua de estados vía WebSockets (SignalR) tolerante a reconexiones, eventos duplicados y consumo at-least-once.
- **Escalabilidad y mantenibilidad**  
Separación clara entre lógica de dominio, infraestructura, casos de uso y capa de presentación visual.
- **Desempeño y concurrencia**  
Soporte para transiciones de estado de partida altamente eficientes y control estricto de accesos concurrentes a la base de datos.

## Decisión

Se define una arquitectura distribuida desacoplada compuesta por un backend sobre **.NET 10** y un frontend sobre **Angular 20**, estructurada bajo los siguientes lineamientos técnicos:

### Backend (.NET 10)

- **Clean Architecture + Domain-Driven Design (DDD) con Vertical Slices**  
Separación modular en Domain, Application, Infrastructure y Api. Los casos de uso se organizan mediante Vertical Slices aislados.
- **Patrón Mediator e Inyección Pipeline**  
Mediador customizado sin dependencias externas pesadas, soportado por un pipeline de comportamientos encadenados (**Logging**, **Validation**, **Idempotency**, **UnitOfWork**).
- **Persistencia y Concurrencia**  
EF Core 10 sobre SQL Server 2022 utilizando **DbContextFactory** para aislar los ciclos de vida de conexiones SignalR de larga duración. Control de concurrencia optimista implementado mediante **RowVersion**.
- **Resiliencia e Idempotencia**  
**Patrón Outbox** para garantizar entrega de eventos *at-least-once*, resiliencia mediante Polly v8 (**Jitter**, **Circuit Breaker**) e idempotencia estricta en endpoints y hubs respaldada por **RequestId**.

## Frontend (Angular 20)

- **Arquitectura Hexagonal en Cliente**
  - ✓ **domain/**  
Reglas de presentación y contratos en `TypeScript` puro (0 dependencias de `@angular/*`, `rxjs` o `@ngrx/*`).
  - ✓ **data-access/**  
`DTOs`, `Mappers` (frontera anticorrupción) y repositorios `HTTP/SignalR`.
  - ✓ **features/**  
Módulos visuales aislados e importados exclusivamente con `lazy loading`.
- **Gestión de Estado Reactiva**  
**NgRx Signal Store** con arquitectura basada en eventos. El store refleja el estado que transmite el backend y abstiene explícitamente cualquier cálculo de negocio o acumulados.
- **Estrategia de Renderizado y Rendimiento**  
Configuración `Zoneless` (`provideZonelessChangeDetection`), componentes `Standalone`, detección de cambios `OnPush` nativa con `Signals` y UI basada en el preset `Aura de PrimeNG` con soporte para tema oscuro mediante `.app-dark`.

## Pruebas y Calidad

Mantenimiento del estándar **AAA (Arrange-Act-Assert)**. Cobertura de reglas puras de dominio sin `TestBed`, fakes en memoria para servicios e infraestructura, y pruebas unitarias aisladas en frontend (`Vitest`) y backend (`xUnit` + `NSubstitute`).

## Alternativas consideradas

| OPCIÓN                                                                                                | ¿POR QUÉ SE DESCARTÓ?                                                                                                                                                                                                                                                                                                                                           |
|-------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Monolito Tradicional MVC o Blazor Server                                                              | Aunque simplifica el despliegue inicial, acopla fuertemente el renderizado con el servidor, consumiendo recursos elevados por conexión abierta en Blazor Server o perdiendo la experiencia fluida y reactiva en tiempo real de una <code>Single Page Application (SPA)</code> dedicada.                                                                         |
| Frontend en Angular con <code>Zone.js</code> y <code>Redux Clásico</code> ( <code>NgRx Store</code> ) | <code>Zone.js</code> introduce sobrecostos en la detección de cambios al interceptar callbacks asíncronos globales (como las conexiones de <code>SignalR</code> ). <code>NgRx Signal Store</code> junto con <code>Zoneless</code> ofrece un modelo mucho más ligero, predictivo y alineado a las APIs modernas del ecosistema <code>TypeScript/Angular</code> . |
| Lógica de Juego Duplicada en Frontend (Cálculo de Premios Local)                                      | Promueve inconsistencias de estado entre cliente/servidor y expone el sistema a vulnerabilidades donde el cliente puede manipular el avance o los acumulados de las rondas.                                                                                                                                                                                     |

## Consecuencias

### Positivas

- **Testabilidad Superior**  
La capa `domain/` del frontend al ser `TypeScript` puro se ejecuta en milisegundos en `Vitest` sin instanciar `TestBed`. En el backend, las reglas de dominio se testean de forma totalmente aislada.

- **Mantenibilidad y Mapeo Transparente**  
La simetría de eventos entre Backend (SignalR Hub) y Frontend (Signal Store) facilita la trazabilidad de bugs y evolutivos.
- **Eficiencia en Runtime**  
Zoneless elimina la sobrecarga de simular ejecución dentro de la zona de Angular ante tráfico constante de WebSockets.
- **Resiliencia Operativa**  
La arquitectura tolera pérdida temporal de red, reintentos de mensajes y cierres inesperados mediante transacciones idempotentes, temporizadores de respaldo y Outbox Pattern.

## Negativas / limitaciones aceptadas

- **Complejidad Inicial de Estructura**  
Requiere mayor disciplina en la separación de capas (Mappers, DTOs, Contratos, Handlers) comparado con soluciones CRUD directas.
- **Costo de Serialización**  
La transformación constante DTO – Domain Model tanto en backend como en frontend añade una leve sobrecarga, aceptada a cambio de blindar el dominio interno de cambios externos.

## Plan de evolución (v1 → v2 / producción)

| Fase                      | Métrica / Criterio de Aceptación                                                                                                              | Herramienta / Método                                           |
|---------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------|
| v1 (Desarrollo / Testing) | Más del 80% de pruebas unitarias exitosas en backend ( <i>dotnet test</i> ) y frontend ( <i>npm run test</i> )                                | xUnit, Vitest                                                  |
| v1 (Calidad de Código)    | Cobertura > 90% en dominios y mappers. Cero violaciones de límites de capa en linter.                                                         | Coverage v8, ESLint (no-restricted-imports)                    |
| v1 (Bundle & Performance) | Tiempos de compilación limpios, cero advertencias ( <i>TreatWarningsAsErrors=true</i> ), lazy loading verificado sin fugas en bundle inicial. | Angular Build Analyzer                                         |
| v2 (Staging / Pre-prod)   | Tolerancia a fallos bajo desconexiones abruptas simuladas de SignalR y reintentos idempotentes.                                               | Integration & Stress Tests                                     |
| v2 (Producción)           | Latencia < 100 ms en entrega de eventos de ronda y 0% de inconsistencias en acumulados de partidas.                                           | APM (Application Performance Monitoring), Logs correlacionados |

## Diagramas

### Modelo de Casos de Uso y Reglas de Negocio

(ver diagrama)

El sistema delimita claramente las responsabilidades de sus tres actores principales mediante un diseño desacoplado de canales de interacción:

- **Administrator (User)**  
Gestiona las entidades del catálogo (Categorías y Preguntas) e interactúa vía peticiones REST/HTTP transaccionales.
- **Player (User)**  
Ejecuta el ciclo interactivo de juego con baja latencia mediante HTTP para la inicialización y WebSockets (SignalR) para las respuestas y eventos.

- **System Timer (Servicio Autónomo)**

Un **BackgroundService** supervisor (**GameTimeoutService**) que vigila activamente los límites de tiempo (30 segundos) y fuerza el fin de la partida si se excede la fecha límite (**DeadlineUtc**).

## Detalle de los Casos de Uso

### Start Game (Iniciar Partida)

Al menos 1 categoría activa por nivel de dificultad configurado, cada una con mínimo 5 preguntas activas.

1. El jugador ingresa su alias en el formulario inicial y solicita iniciar partida.
2. La SPA envía **POST /api/games**. El backend valida (2 a 60 caracteres) mediante el **Value Object** **PlayerName**.
3. Se inicializa el agregado Game en estado **InProgress**, asignando una pregunta aleatoria de dificultad 1.
4. Se persiste la transacción unitaria junto con los eventos **GameStartedDomainEvent** y **RoundStartedDomainEvent** en la tabla **OutboxMessages**.
5. La SPA se suscribe al canal **SignalR /hubs/game** mediante el **gameld** y comienza la cuenta regresiva.

### Answer Question (Responder Pregunta)

Partida en estado **InProgress** con ronda abierta dentro del plazo estipulado.

#### Respuesta Correcta:

1. El jugador selecciona una opción. La SPA invoca el método **SignalR SubmitAnswerAsync** con un **requestId** único para garantizar idempotencia.
2. El **IdempotencyBehavior** verifica si la solicitud ya fue procesada; si es así, retorna el resultado previo almacenado sin re-ejecutar.
3. **AnswerQuestionCommandHandler** evalúa la respuesta en el dominio con bloqueo de concurrencia optimista (**RowVersion**).
4. Si es correcta, la ronda se marca como **RoundOutcome.Correct**, se acumula el premio en la estrategia correspondiente y se avanza de nivel.
5. Se guardan los cambios y el **OutboxProcessor** notifica a los clientes vía SignalR de forma estrictamente posterior al commit.

#### Respuesta Incorrecta:

Se marca la ronda como **RoundOutcome.Incorrect**, el juego pasa a estado terminal **Lost** y el premio acumulado se reinicia a \$0 COP.

### Withdraw From Game (Retiro Voluntario)

Partida en **InProgress**, ronda actual sin responder y acumulado mayor a 0.

1. El jugador presiona "Retirarse". Se despliega un diálogo de confirmación para asegurar la decisión.
2. Al confirmar, la SPA invoca **WithdrawAsync**. El backend ejecuta **WithdrawGameCommand**.
3. El agregado Game realiza la transición **InProgress** -> **Withdrawn** y fija **EndedAtUtc**.
4. Se asegura el premio acumulado obtenido hasta la ronda anterior y se redirige a la pantalla de resultados.

## Arquitectura del Sistema (C4)

(ver diagrama)

La solución se estructura bajo el estándar C4 para representar de forma clara la distribución de contenedores, la separación de responsabilidades.

## Contenedores y Despliegue (C4 Nivel 1 y 2)

(ver diagrama)

El sistema se distribuye en cuatro contenedores principales totalmente aislados:

- **Web Server & Reverse Proxy (Nginx Alpine)**  
Centraliza el tráfico en el puerto `443/80`, sirve los estáticos de Angular y realiza enrutamiento de peticiones `REST (/api/*)` y negociación de `WebSockets (/hubs/*)`.
- **Single Page Application (Angular 20)**  
Cliente SPA reactivo sin zona (Zoneless), con arquitectura modular `signalStore` y componentes `PrimeNG Aura`.
- **API Backend (.NET 10)**  
`Minimal APIs` modulares, controladores de WebSockets con `SignalR Core`, supervisores de fondo y lógica de dominio rica en C# 13.
- **Database (SQL Server 2022)**  
Persistencia relacional ACID, soporte de concurrencia optimista, tablas de idempotencia y almacenamiento de outbox.

## Ciclo de Vida y Máquina de Estados

(ver diagrama)

El juego se rige por una máquina de estados determinista y fuertemente tipada (`GameStateMachine`).

Transiciones de Estado

| Estado Origen | Estado Destino | Disparador/Evento | Condición de Guarda                        | Termina |
|---------------|----------------|-------------------|--------------------------------------------|---------|
| NotStarted    | InProgress     | Game.Create()     | Settings válidos y alias correcto.         | No      |
| InProgress    | InProgress     | Game.Answer()     | Respuesta correcta y ronda < total.        | No      |
| InProgress    | Won            | Game.Answer()     | Respuesta correcta y ronda == total (5/5). | Si      |
| InProgress    | Lost           | Game.Answer()     | Respuesta incorrecta (premio pasa a \$0).  | Si      |
| InProgress    | Withdrawn      | Game.Withdraw()   | Retiro voluntario (conserva acumulada).    | Si      |
| InProgress    | ForcedEnd      | Game.ForceEnd()   | Tiempo límite expirado (30s).              | Si      |

Una vez que una partida alcanza un estado terminal (`Won`, `Lost`, `Withdrawn`, o `ForcedEnd`), no existen transiciones salientes permitidas. Cualquier intento posterior de modificar la partida es rechazado inmediatamente en el dominio.

## Modelo Relacional de Datos (SQL Server 2022)

(ver diagrama)

El esquema fue diseñado para garantizar aislamiento `ACID`, prevenir condiciones de carrera mediante concurrencia optimista y soportar el patrón `Transactional Outbox`.

## Diccionario de Datos

| Tabla      | Columna                                                  | Tipo SQL                                                         | Restricción            | Descripción                                              |
|------------|----------------------------------------------------------|------------------------------------------------------------------|------------------------|----------------------------------------------------------|
| Categories | Id<br>Name<br>DifficultyLevel<br>PrizeAmount<br>IsActive | uniqueidentifier<br>nvarchar(100)<br>int<br>decimal(18,2)<br>bit | PK, UQ_Categories_Name | Catálogo de temas por nivel de dificultad y premio base. |

|                           |                                                                                                       |                                                                                                                  |                                                 |                                                                               |
|---------------------------|-------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------|-------------------------------------------------|-------------------------------------------------------------------------------|
| <b>Questions</b>          | Id<br>CategoryId<br>Text<br>IsActive                                                                  | uniqueidentifier<br>uniqueidentifier<br>nvarchar(500)<br>bit                                                     | PK, FK (Categories)<br>RESTRICT                 | Lista de preguntas por categoría. Indexado por <b>CategoryId + IsActive</b> . |
| <b>Answers</b>            | Id<br>QuestionId<br>Text<br>IsCorrect                                                                 | uniqueidentifier<br>uniqueidentifier<br>nvarchar(300)<br>bit                                                     | PK, FK (Questions) CASCADE                      | Opciones de respuesta (exactamente 4 por pregunta, 1 correcta).               |
| <b>Games</b>              | Id<br>PlayerName<br>Status<br>CurrentRound<br>AccumulatedPrize<br>RowVersion                          | uniqueidentifier<br>nvarchar(60)<br>nvarchar(20)<br>int<br>decimal(18,2)<br>rowversion                           | PK, RowVersion (Optimistic Lock)                | Cabecera de partida. Mantiene el token de concurrencia optimista.             |
| <b>Rounds</b>             | Id<br>GameId<br>Number<br>QuestionId<br>CorrectAnswerId<br>SelectedAnswerId<br>Outcome<br>DeadlineUtc | uniqueidentifier<br>uniqueidentifier<br>int<br>uniqueidentifier<br>uniqueidentifier<br>nvarchar(20)<br>datetime2 | PK, FK (Games) CASCADE, UQ_Rounds_GameId_Number | Detalle de rondas ejecutadas y límite de tiempo inmutable.                    |
| <b>OutboxMessages</b>     | Id<br>OccurredOnUtc<br>Type<br>Content<br>ProcessedOnUtc<br>AttemptCount                              | uniqueidentifier<br>datetime2<br>nvarchar(400)<br>nvarchar(4000)<br>datetime2<br>int                             | PK, IX_Outbox_Pending                           | Cola transaccional de eventos de dominio para notificación por SignalR.       |
| <b>IdempotencyRecords</b> | RequestId<br>CommandName<br>StatusCode<br>ResponsePayload<br>ExpiresOnUtc                             | uniqueidentifier<br>nvarchar(200)<br>int<br>nvarchar(4000)<br>datetime2                                          | PK                                              | Almacén de peticiones procesadas para neutralizar duplicidad.                 |

## Invariantes de Dominio vs Restricciones de Esquema

Para mantener un modelo flexible y de alto rendimiento, ciertas reglas de negocio complejas se garantizan estrictamente en la capa de Dominio (DDD) en lugar de usar disparadores o restricciones SQL complejas:

- **Exactamente 4 opciones por pregunta y 1 correcta**  
Validado en memoria dentro de `Question.Create()` antes de llamar a la base de datos.
- **Mínimo 5 preguntas activas para activar categoría**  
Verificado en el método de negocio `Category.Activate()`.
- **No repetir preguntas en la misma partida**  
Comprobado en `Game.AssignQuestion()` utilizando la colección `AskedQuestionIds`.
- **Cálculo de pérdidas en fallos**  
Manejado en la función pura `GameRules.AccumulateFor()`.

## Referencias

- **Microsoft Docs**  
[\*Architecting Modern Web Applications with ASP.NET Core and Azure \(Clean Architecture & DDD patterns\)\*](#)
- **Angular Architecture Guidelines**  
[\*Zoneless Applications & Signal-based State Management\*](#)
- **Event-Driven Architecture Standards**  
[\*Outbox Pattern & At-Least-Once Delivery Guarantees\*](#)
- **RFC 9457**  
[\*Problem Details for HTTP APIs\*](#)